

LAB 9

Operator Overloading

Learning Objectives

By the end of this lab, you should be able to:

- Explain what operator overloading is and why it makes code more natural to read.
- Write an overload of a binary operator (+, −) for one of your own classes.
- Distinguish unary operators (++ , --) from binary operators (+, −, ==).
- Tell the difference between returning a NEW object (for +, −) and returning a reference to *this (for +=, -=).
- Overload prefix ++ and postfix ++ separately, using the dummy int parameter trick.
- Overload comparison operators (>, <, ==) that return bool.

1. What Is Operator Overloading?

Operator overloading is a C++ feature that lets you teach built-in symbols (+, −, ==, ++, and many more) how to work on objects of your own classes — just like they already work on int, float, and so on.



The syntax-sugar translation

```

      obj1 + obj2
        |
    |   compiler rewrites it as:
        v
    obj1.operator+(obj2)
  
```

Without operator overloading, you would have to write something ugly like `c3 = c1.add(c2);`. With operator overloading, you can write `c3 = c1 + c2;` — which reads exactly like the math you already know. That is the whole point.

Syntax

```

return_type operator<symbol>(parameters) {
    // logic
}
  
```

```
}
```

Two Categories

- **Unary operators** — work on a single operand. Examples: `++`, `--`, `-` (negation).
- **Binary operators** — work on two operands. Examples: `+`, `-`, `*`, `/`, `==`, `+=`, `-=`.

A few operators you CAN'T overload

C++ lets you overload most operators, but not all. The ones that are **off-limits** include: `::` (scope resolution), `.` (member access), `.*` (pointer-to-member), `?:` (ternary), and `sizeof`. Everything else is fair game.

2. Code 1 — One Operator, Many Meanings

Goal: see that the SAME operator (– and +) can be overloaded for many different classes, each with its own meaning. We'll define three small classes — Text, complex, and vector — and teach – and + to behave sensibly for each.

The Complete Code

```
#include <iostream>
using namespace std;

// ----- Text class -----
class Text {
public:
    string str;
    Text(string s) { str = s; }

    // Subtract last n characters from the string
    Text operator-(int n) {
        string res = str;
        res = res.substr(0, res.length() - n);
        return Text(res);
    }
};

// ----- complex class -----
class complex {
public:
    int real, img;
    complex(int r = 0, int i = 0) { real = r; img = i; }

    complex operator-(complex c) {
        complex temp;
        temp.real = real - c.real;
        temp.img = img - c.img;
        return temp;
    }
    complex operator+(complex c) {
        complex temp;
        temp.real = real + c.real;
        temp.img = img + c.img;
        return temp;
    }
};

// ----- vector class -----
class vector {
```

```

public:
    int i, j, k;
    vector(int a = 0, int b = 0, int c = 0) { i = a; j = b; k = c; }

    vector operator-(vector v) {
        vector temp;
        temp.i = i - v.i;
        temp.j = j - v.j;
        temp.k = k - v.k;
        return temp;
    }
    vector operator+(vector v) {
        vector temp;
        temp.i = i + v.i;
        temp.j = j + v.j;
        temp.k = k + v.k;
        return temp;
    }
};

int main() {
    // Built-in int subtraction and addition (no overload needed)
    int a = 10, b = 5;
    cout << (a - b) << " " << (a + b) << endl;

    // Text: subtract last 5 characters
    Text s("Hello World!");
    Text s2 = s - 5;
    cout << s2.str << endl;

    // complex: subtract and add two complex numbers
    complex c1(6, 6), c2(3, 4);
    complex c3 = c1 - c2;
    cout << c3.real << " + " << c3.img << "i" << endl;
    complex c4 = c1 + c2;
    cout << c4.real << " + " << c4.img << "i" << endl;

    // vector: subtract and add two 3D vectors
    vector v1(7, 8, 9), v2(2, 2, 2);
    vector v3 = v1 - v2;
    cout << v3.i << "i + " << v3.j << "j + " << v3.k << "k" << endl;
    vector v4 = v1 + v2;
    cout << v4.i << "i + " << v4.j << "j + " << v4.k << "k" << endl;

    return 0;
}

```

Code Breakdown — The Three Overloads Side by Side

Code	What it does
Text operator-(int n)	← Takes an int on the right side. Truncates the string by n characters from the end. Returns a NEW Text.
complex operator-(complex c)	← Takes another complex on the right. Subtracts real-from-real and img-from-img. Returns a NEW complex.
vector operator-(vector v)	← Takes another vector on the right. Subtracts component by component. Returns a NEW vector.
operator+(...)	← Same pattern as – but for addition. Notice: NO operator+ was written for Text — so s + 5 would NOT compile.
s - 5	← Compiler sees: "left side is Text, right side is int" → picks Text::operator-(int).
c1 - c2	← Compiler sees: "left side is complex, right side is complex" → picks complex::operator-(complex).

Expected Output

► Output

```

5 15
Hello W
3 + 2i
9 + 10i
5i + 6j + 7k
9i + 10j + 11k

```

**The compiler picks the right overload automatically**

There is no "global" minus operator that handles every class. Each class defines its own. The compiler simply looks at the **type of the operands** and chooses the matching overload — exactly the same way it picks between *search(int)* and *search(string)* in function overloading.

**Why we return a NEW object (not a reference) here**

Operators like `+` and `-` should produce a **new value** without modifying the original operands. $a + b$ does not change a or b — it creates a third value. That is why these overloads build a *temp* object and return it. The next code example shows the OPPOSITE case (`+=` modifies in place), so the contrast will be clear.

3. Code 2 — New Object vs Reference (+ vs +=)

Goal: see the crucial difference between operators that RETURN A NEW OBJECT and operators that MODIFY THE SAME OBJECT. The BankAccount class implements both styles so you can compare them directly.

The Code

```
#include <iostream>
using namespace std;

class BankAccount {
public:
    string name;
    int    balance;

    BankAccount(string name = " ", int balance = 0) {
        this->name = name;
        this->balance = balance;
    }

    // --- Returns a NEW object ---
    BankAccount operator-(int amount) {
        if (amount > balance) {
            cout << "Insufficient balance! Can't Withdraw." << endl;
            return BankAccount(name, balance);           // unchanged copy
        }
        return BankAccount(name, balance - amount);      // new account, less money
    }

    BankAccount operator+(int amount) {
        return BankAccount(name, balance + amount);
    }

    // --- Modifies THE SAME object, returns a reference to it ---
    BankAccount& operator-=(int amount) {
        if (balance < amount)
            cout << "Insufficient balance! Can't Withdraw." << endl;
        else
            balance -= amount;
        return *this;                                     // return the modified object
    }

    BankAccount& operator+=(int amount) {
        balance += amount;
        return *this;
    }

    void show() {
        cout << "Name: " << name << "\t Balance: " << balance << endl;
    }
};
```

```

int main() {
    BankAccount acc("Alifa", 500);
    acc.show();

    acc = acc - 300;           // creates a NEW object, then assigns it back to acc
    acc.show();

    acc = acc - 1200;         // fails – insufficient balance
    acc.show();

    acc -= 100;               // MODIFIES acc in place
    acc.show();

    acc -= 1000;              // fails – insufficient balance
    acc.show();

    acc = acc + 200;          // creates a NEW object
    acc.show();

    acc += 300;               // MODIFIES acc in place
    acc.show();

    return 0;
}

```

The Critical Difference

Code	What it does
BankAccount operator-(int amount)	← Return type is BankAccount (by value). Builds a brand new BankAccount and returns it. The original is untouched.
BankAccount& operator-=(int amount)	← Return type is BankAccount& (a reference). Modifies the original via balance -= amount, then returns *this — a reference to the same object that was just changed.
acc = acc - 300;	← Step 1: operator-(300) builds a new BankAccount. Step 2: that new object is assigned BACK to acc. Two steps; an extra temporary is created.
acc -= 100;	← ONE step. operator-=(100) directly changes acc's balance. No temporary, no assignment. Faster and cleaner.
return *this;	← this is a pointer to the current object; *this is the object itself. We return a reference to it so chaining works (e.g., (acc += 10) += 20;).

Expected Output

► Output

```
Name: Alifa      Balance: 500
Name: Alifa      Balance: 200
Insufficient balance! Can't Withdraw.
Name: Alifa      Balance: 200
Name: Alifa      Balance: 100
Insufficient balance! Can't Withdraw.
Name: Alifa      Balance: 100
Name: Alifa      Balance: 300
Name: Alifa      Balance: 600
```



The rule of thumb — when to return what

If the operator is **arithmetic** (+, -, *, /) → return a NEW object by VALUE. The original operands must not be changed.

If the operator is **compound assignment** (+=, -=, *=, /=) → MODIFY the current object in place and return a REFERENCE to it (return *this;).

If the operator is a **comparison** (==, <, >) → return *bool* (coming up in Code 4).

4. Code 3 — Prefix vs Postfix (++ and --)

Goal: understand why C++ needs TWO different overloads for the same ++ symbol — one for prefix (++w) and one for postfix (w++). The behaviors are subtly different and so are the function signatures.



Prefix vs postfix — the behavioral difference

Suppose w.kg starts at 10.

```
int x = ++w.kg;    // PREFIX: increase first, THEN return
                  // → w.kg = 11, x = 11
```

```
int y = w.kg++;    // POSTFIX: return first, THEN increase
                  // → y = 11, w.kg = 12
```

The Code

```
#include <iostream>
using namespace std;

class Weight {
public:
    int kg;
    Weight(int k = 0) { kg = k; }

    // ----- Prefix ++w -----
    // No parameters. Increases first, returns reference to *this.
    Weight& operator++() {
        kg = kg + 1;
        return *this;
    }

    // ----- Postfix w++ -----
    // Takes a DUMMY int parameter (just to distinguish the signature).
    // Saves the old value, then increments, then returns the OLD value (by value).
    Weight operator++(int) {
        Weight temp = *this;
        kg = kg + 1;
        return temp;
    }

    // Same pattern for --
    Weight& operator--() {
        kg = kg - 1;
        return *this;
    }
    Weight operator--(int) {
        Weight temp = *this;
        kg = kg - 1;
        return temp;
    }
}
```

```

};

int main() {
    Weight w(10), w2, w3;

    w2 = w++;      // POSTFIX: w2 gets the OLD value (10), then w becomes 11
    w3 = ++w;      // PREFIX: w becomes 12, w3 gets the NEW value (12)

    cout << "After prefix & postfix ++:" << endl;
    cout << w.kg << endl << w2.kg << endl << w3.kg << endl;

    w2 = w--;      // POSTFIX --: w2 gets the OLD value (12), then w becomes 11
    w3 = --w;      // PREFIX --: w becomes 10, w3 gets the NEW value (10)

    cout << "After prefix & postfix --:" << endl;
    cout << w.kg << endl << w2.kg << endl << w3.kg << endl;
}

```

Code Breakdown

Code	What it does
Weight& operator++()	← PREFIX version — no parameters. Modifies the object and returns a reference to it. Reference return → no extra copy.
Weight operator++(int)	← POSTFIX version — has a DUMMY int parameter (unused — it just makes the signature different from prefix). Returns the OLD value BY VALUE.
Weight temp = *this;	← In postfix, we MUST save the old state BEFORE incrementing. That saved temp is what we return.
w2 = w++;	← Calls postfix operator++(int). w2 ends up with the value w HAD before the increment.
w3 = ++w;	← Calls prefix operator++(). w3 ends up with the value w HAS after the increment.
operator-- (both versions)	← Mirror image of ++. The int parameter trick is the same way C++ distinguishes prefix -- from postfix --.

Expected Output

► Output

```

After prefix & postfix ++:
12
10
12
After prefix & postfix --:
10

```

```
12  
10
```

💡 Why the dummy `int` parameter?

Both `++w` and `w++` look like "the same operator with one operand," so C++ needed SOME way to tell them apart at the function level. The designers picked a clever hack: postfix takes an unused `int` parameter. The value passed (always 0) is never used — it exists purely so the compiler can tell the two overloads apart by signature.

⚠️ Always prefer prefix in your own code

Postfix has to create and return a temporary copy of the old value — that costs a little extra work. Prefix just modifies and returns **this*. For built-in types like `int` the difference is invisible, but for big objects it can matter. **Use prefix unless you specifically need the old value.**

5. Code 4 — Comparison Operators (>, <, ==)

Goal: overload comparison operators that return bool. These let you compare two objects of your class as naturally as you compare two integers.

The Code

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int    marks;

    Student(string n = "", int m = 0) {
        name  = n;
        marks = m;
    }

    bool operator>(Student s) { return marks > s.marks; }
    bool operator<(Student s) { return marks < s.marks; }
    bool operator==(Student s) { return marks == s.marks; }

    void show() {
        cout << "Name: " << name << "\t Marks: " << marks << endl;
    }
};

int main() {
    Student s1("Alice", 85);
    Student s2("Bob", 90);
    Student s3("Coco", 85);

    s1.show();
    s2.show();
    s3.show();
    cout << endl;

    if (s1 > s2)
        cout << s1.name << " has more marks than " << s2.name << endl;
    else
        cout << s2.name << " has more marks than " << s1.name << endl;

    if (s1 < s2)
        cout << s1.name << " has less marks than " << s2.name << endl;

    if (s1 == s3)
        cout << s1.name << " and " << s3.name << " have equal marks" << endl;
}
```

Code Breakdown

Code	What it does
<code>bool operator>(Student s)</code>	← Returns true if THIS student has more marks than the other. Return type is bool because the result is a yes/no answer.
<code>return marks > s.marks;</code>	← Reduce the object comparison to a simple int comparison on whichever field defines the ordering (here, marks).
<code>if (s1 > s2)</code>	← Compiler rewrites this as <code>if (s1.operator>(s2))</code> — and uses the returned bool to drive the if.
<code>operator< and operator==</code>	← Same pattern — each compares the marks field and returns a bool.

Expected Output

► Output

```
Name: Alice      Marks: 85
Name: Bob       Marks: 90
Name: Coco      Marks: 85
```

```
Bob has more marks than Alice
Alice has less marks than Bob
Alice and Coco have equal marks
```



Pick a meaningful field to compare on

Here we compared Students by marks because that's what "greater than" naturally means for students. For a Book class, you might compare by price; for a Date class, by year/month/day. The point is that YOU decide what the comparison means for your class — the compiler is happy to use whatever logic you write.

6. Quick Reference — Return Types at a Glance

Use this table as a cheat sheet whenever you write a new operator overload:

Operator type	Examples	Return type	Modifies operands?
Arithmetic	+, -, *, /	NEW object (by value)	No
Compound assign	+=, -=, *=, /=	Reference (T&) to *this	Yes (modifies left operand)
Unary prefix	++w, --w	Reference (T&) to *this	Yes
Unary postfix	w++, w--	Old value (by value)	Yes (after returning)
Comparison	>, <, ==, !=, >=, <=	bool	No

7. Summary

Six things to take away from this lab:

- **Operator overloading is just function overloading** with a special name — the function is named *operator+*, *operator-*, and so on.
- **obj1 + obj2 is compiler shorthand** for *obj1.operator+(obj2)*. The compiler picks the overload by looking at the types of both operands.
- **Arithmetic operators return a NEW object** (by value). Compound-assignment operators MODIFY the same object and return a REFERENCE.
- **Prefix and postfix ++ are TWO different overloads.** Postfix takes a dummy *int* parameter to distinguish itself.
- **Comparison operators return bool**, not the class type. Pick whichever field naturally defines the ordering.
- **Most operators can be overloaded**, but a handful (*::*, *.*, ***, *?:*, *sizeof*) cannot. Almost everything else is fair game.

8. Practice Tasks

Task 1 — Text Class with + and –

Create a class Text with a single string data member.

Requirements

- Overload the + operator so that *t1 + t2* returns a new Text whose string is the concatenation of the two.

- Overload the `-` operator so that `t - n` returns a new `Text` with the last `n` characters removed (`n` is an `int`).
- Create at least three `Text` objects in `main` and demonstrate both operations.
- Display the resulting string after each operation.

Hint for Task 1

For concatenation: `return Text(str + t.str);`. For truncation: use `str.substr(0, str.length() - n)`. This task is excellent practice because the SAME symbol (`-`) takes a `Text` on the left but an `int` on the right.

Task 2 — Distance Class (feet, inches)

Create a class `Distance` with two `int` data members: `feet` and `inches`.

Requirements

- Overload `+` so that `d1 + d2` returns a new `Distance` with the sum, correctly carrying over when `inches ≥ 12` (e.g., 5 ft 8 in + 2 ft 7 in = 8 ft 3 in).
- Overload `-` to subtract one `Distance` from another. Handle the case where you need to borrow 12 inches from the feet column.
- If the subtraction would produce a negative distance, print an error and return a `Distance` with `feet = 0` and `inches = 0` (or any other sensible default).
- Provide a `show()` function to display in the form "X feet Y inches".

Hint for Task 2

Add inch-by-inch and foot-by-foot first, then normalize: *if (`inches >= 12`) { `feet += inches / 12`; `inches %= 12`; }*. For subtraction, *if (`inches < 0`) { `feet--`; `inches += 12`; }* — the classic borrowing trick.

Task 3 — Counter Class with ++ (Prefix and Postfix)

Create a class `Counter` with a single `int` data member `count`.

Requirements

- Overload the prefix `++` operator so that `++c` increases `count` by 1 and returns the updated counter.
- Overload the postfix `++` operator so that `c++` increases `count` by 1 but returns the OLD value.
- Demonstrate the difference clearly in `main` by assigning the results of `c++` and `++c` to two different counters and printing all the values.

What you should observe

Start with `Counter c(5);`. Then `Counter a = c++;` gives `a.count = 5` and `c.count = 6`. Then `Counter b = ++c;` gives `b.count = 7` and `c.count = 7`. If your output matches that, both overloads are working correctly.

Prepared by

Alifa Shanzidah Manarat

Lecturer

Dept. of CSE, BAUST